

Ben Gurion University of the Negev
Faculty of Engineering Sciences
Mechanical Engineering Department

Three Degrees of Freedom Robotic Arm

Robots Motion Planning and Control Final Project

Written by: Sharon Guberg

Nave Markovich

Bar Mizrachi

Link list:

1. [Full Drive folder](#)
2. [ReadMe](#)
3. Zero input simulation
 - 3.1. [Section 4-6 MATLAB](#)
 - 3.2. **Videos:**
 - 3.2.1. [First angles positions](#) [20 120 -70]
 - 3.2.2. [Second angles positions](#) [0 250 30]
 - 3.2.3. [Third angles positions](#) [0 90 0]
 - 3.2.4. [Fourth angles positions](#) [0 270 180]
4. Controlled simulation
 - 4.1. [Section 7-9 MATLAB](#)
 - 4.2. **Videos:**
 - 4.2.1. [Start \[-50 -50 -20\] End \[0.25 0.45 0.20\]](#) (default point)
 - 4.2.2. [Start \[50 50 -20\] End \[0.25 0.45 0.20\]](#)
 - 4.2.3. [Start \[10 90 -20\] End \[0.25 0.45 0.20\]](#)
 - 4.2.4. [Start \[50 50 -20\] End \[-0.40 0.20 0.10\]](#)
 - 4.2.5. [Start \[50 50 -20\] End \[0.00 0.46 0.00\]](#)
 - 4.2.6. Op.1 [Start \[50 50 -20\] End \[-0.20 -0.40 -0.01\]](#),
 - 4.2.7. Op.2 [Top View](#)
 - 4.2.8. Op.3 [Different angle](#)
5. [Full Kinematics](#) (this is the Mathematica file)

1. Introduction

The objective of this part of the mini project is to model, simulate, and control a robotic manipulator with at least three degrees of freedom. In this work, a three-degree-of-freedom robotic arm is selected and represented in Figure 1. The robot is modeled as a serial manipulator composed of three rigid links connected by three revolute joints. Although real industrial robotic arms are usually more complex and may include additional degrees of freedom, the 3-DOF model captures the main dynamic behavior of a multi-link robotic manipulator.

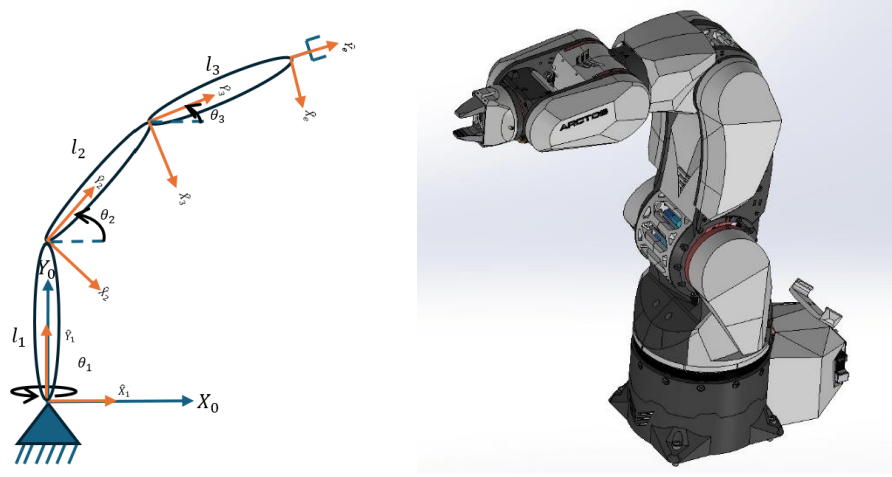


Figure 1: Simplified 3-DOF Robot Arm Model and Its Industrial Robot Analogy

The configuration of the robot is described by the generalized coordinates

$$q(t) = \begin{bmatrix} \theta_1(t) \\ \theta_2(t) \\ \theta_3(t) \end{bmatrix} \quad (1.1)$$

where $(\theta_1, \theta_2, \theta_3)$ are the time dependent joint angles of the robot arm. The robot consists of three rigid links with lengths (l_1, l_2, l_3) . Joint l_1 is rotate around the $\hat{y}_1$ and l_2, l_3 rotate around there own $\hat{z}$ axis. In this work, we first present a simulation that investigates the natural dynamics of the system in the absence of control input. Subsequently, we design a controller capable of driving the robot from an initial configuration to a desired target configuration.

2. Kinematic and Dynamic Model of the Robot

The objective of this chapter is to derive the mathematical model used for the robot simulation and control. The kinematic part is presented only as the basis for the dynamic derivation. Therefore, the derivation focuses on the center-of-mass positions, the linear Jacobians, and the angular Jacobians of the links.

Note:

For this report, we use the short notation:

$$c_i = \cos(\theta_i), s_i = \sin(\theta_i)$$

and for compact notation,

$$c_{12} = \cos(\theta_1 + \theta_2), s_{12} = \sin(\theta_1 + \theta_2)$$

$$c_{123} = \cos(\theta_1 + \theta_2 + \theta_3), s_{123} = \sin(\theta_1 + \theta_2 + \theta_3)$$

2.1. Center-of-Mass Positions

The dynamic model is derived from the motion of each link center of mass. The position of the center of mass of link 1 is

$$r_{c1} = \begin{bmatrix} 0 \\ l_{c1} \\ 0 \end{bmatrix} \quad (2.1)$$

The center of mass of link 2 is

$$r_{c2} = \begin{bmatrix} l_{c2}c_1c_2 \\ l_1 + l_{c2}s_2 \\ l_{c2}s_1c_2 \end{bmatrix} \quad (2.2)$$

The center of mass of link 3 is

$$r_{c3} = \begin{bmatrix} c_1(l_2c_2 + l_{c3}c_{23}) \\ l_1 + l_2s_2 + l_{c3}s_{23} \\ s_1(l_2c_2 + l_{c3}c_{23}) \end{bmatrix} \quad (2.3)$$

These expressions were implemented symbolically in Mathematica and used to calculate the linear velocities of the links.

2.2. Linear Jacobians

The linear velocity of each link center of mass is written as

$$\dot{p}_{ci} = J_{L_i}(q)\dot{q} \quad (2.4)$$

Where

$$J_{L_i}(q) = \frac{\partial p_{ci}}{\partial q} \quad (2.5)$$

After entering it to Mathematica as

```
J11 = Transpose[{D[rc1, q[[1]]], D[rc1, q[[2]]], D[rc1, q[[3]]]}];
J12 = Transpose[{D[rc2, q[[1]]], D[rc2, q[[2]]], D[rc2, q[[3]]]}];
J13 = Transpose[{D[rc3, q[[1]]], D[rc3, q[[2]]], D[rc3, q[[3]]]}];
```

Figure 2: Jacobian of each link defined by the center of mass in Wolfram

We got

$$\begin{aligned} J_{L_1} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} & J_{L_2} &= \begin{bmatrix} -l_{c2}s_1c_2 & -l_{c2}c_1s_2 & 0 \\ 0 & l_{c2}c_2 & 0 \\ l_{c2}c_1c_2 & -l_{c2}s_1s_2 & 0 \end{bmatrix} \\ J_{L_3} &= \begin{bmatrix} -s_1(l_2c_2 + l_{c3}c_{23}) & -c_1(l_2s_2 + l_{c3}s_{23}) & -l_{c3}c_1s_{23} \\ 0 & l_2c_2 + l_{c3}c_{23} & l_{c3}c_{23} \\ c_1(l_2c_2 + l_{c3}c_{23}) & -s_1(l_2s_2 + l_{c3}s_{23}) & -l_{c3}s_1s_{23} \end{bmatrix} \end{aligned} \quad (2.6)$$

2.3. Angular Jacobians

The angular velocity of each link is written as

$$\omega_i = J_{\omega_i}(q)\dot{q} \quad (2.7)$$

The first joint rotates around the fixed Y -axis. The second and third joint axes are rotated by θ_1 , therefore their direction in the base frame is

$$\begin{bmatrix} -s_1 \\ 0 \\ c_1 \end{bmatrix} \quad (2.8)$$

Thus, the angular Jacobians are

$$J_{\omega_1} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} J_{\omega_2} = \begin{bmatrix} 0 & -s_1 & 0 \\ 1 & 0 & 0 \\ 0 & c_1 & 0 \end{bmatrix} J_{\omega_3} = \begin{bmatrix} 0 & -s_1 & -s_1 \\ 1 & 0 & 0 \\ 0 & c_1 & c_1 \end{bmatrix} \quad (2.9)$$

2.4. The Moment of Inertia Matrix

The moment of inertia of each link describes the resistance of the link to angular acceleration around its center of mass. In the dynamic model, each link is treated as a rigid body with a local inertia tensor I_i , defined in the coordinate frame attached to the link.

$$I_1 = \begin{bmatrix} I_{xx1} & 0 & 0 \\ 0 & I_{yy1} & 0 \\ 0 & 0 & I_{zz1} \end{bmatrix} I_2 = \begin{bmatrix} I_{xx2} & 0 & 0 \\ 0 & I_{yy2} & 0 \\ 0 & 0 & I_{zz2} \end{bmatrix} \quad (2.10)$$

$$I_3 = \begin{bmatrix} I_{xx3} & 0 & 0 \\ 0 & I_{yy3} & 0 \\ 0 & 0 & I_{zz3} \end{bmatrix}$$

2.5. The Moment of Inertia Matrix

The rotation matrices of the three joints are

$$R_{01} = \begin{bmatrix} \cos(\theta_1) & 0 & \sin(\theta_1) \\ 0 & 1 & 0 \\ -\sin(\theta_1) & 0 & \cos(\theta_1) \end{bmatrix} R_{12} = \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 \\ \sin(\theta_2) & \cos(\theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.11)$$

$$R_{23} = \begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 \\ \sin(\theta_3) & \cos(\theta_3) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The rotation matrix R_i describes the orientation of link i relative to the fixed base frame. We calculated the orientation of each link with respect to the base frame.

The rotation matrix R_i describes the orientation of link i relative to the fixed base frame. We calculated the orientation of each link with respect to the base frame.

$$R_1 = R_{01} \quad R_2 = R_{01}R_{12} \quad R_3 = R_{01}R_{12}R_{23} \quad (2.12)$$

Therefore, the rotation matrices, using the compact notation, of the three links are

$$R_1 = \begin{bmatrix} c_1 & 0 & -s_1 \\ 0 & 1 & 0 \\ s_1 & 0 & c_1 \end{bmatrix} \quad R_2 = \begin{bmatrix} c_1 c_2 & s_2 & -s_1 c_2 \\ -c_1 s_2 & c_2 & s_1 s_2 \\ s_1 & 0 & c_1 \end{bmatrix} \quad (2.13)$$

$$R_3 = \begin{bmatrix} c_1 c_{23} & s_{23} & -s_1 c_{23} \\ -c_1 s_{23} & c_{23} & s_1 s_{23} \\ s_1 & 0 & c_1 \end{bmatrix}$$

2.6. Equations of Motion

The goal of this part is to obtain the equations of motion of the robot, which describes the relation between the joint motion and the torques applied at the joints.

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \quad (2.14)$$

where $M(q)$ is the inertia matrix of the robot, $C(q, \dot{q})\dot{q}$ represents the Coriolis and centrifugal effects, $G(q)$ is the gravity vector, and τ is the vector of input torques applied at the joints.

2.6.1. Inertia Matrix

After completing the kinematic model of the robot, the next step was to calculate the inertia matrix $M(q)$.

$$M(q) = \sum_{i=1}^3 (m_i J_{L_i}^T J_{L_i} + J_{A_i}^T R_i I_i R_i^T J_{A_i}) \quad (2.15)$$

where J_{L_i} is the linear Jacobian of the center of mass of link i , J_{A_i} is the angular Jacobian of link i , m_i is the mass of link i , R_i is the rotation matrix of link i , and I_i is the inertia tensor of link i in its local coordinate frame.

In Mathematica, the equation was implemented as:

$$\mathbf{M} = \sum_{i=1}^3 (m[\mathbf{i}] \times \text{Transpose}[\mathbf{JL}[\mathbf{i}]] \cdot \mathbf{JL}[\mathbf{i}] + \text{Transpose}[\mathbf{JA}[\mathbf{i}]] \cdot \mathbf{R}[\mathbf{i}] \cdot \mathbf{II}[\mathbf{i}] \cdot \text{Transpose}[\mathbf{R}[\mathbf{i}]] \cdot \mathbf{JA}[\mathbf{i}])$$

Figure 3: Mathematica implementation of the inertia matrix calculation

After simplification, Mathematica returned a symmetric 3×3 matrix

$$\text{'MatrixForm='}$$

$$\begin{pmatrix} \text{Iyy1} + \text{Iyy2} \cos[\theta_2[t]]^2 + \text{lc2}^2 m_2 \cos[\theta_2[t]]^2 + \text{Iyy3} \cos[\theta_2[t] + \theta_3[t]]^2 + m_3 (12 \cos[\theta_2[t]] + \text{lc3} \cos[\theta_2[t] + \theta_3[t]])^2 + \text{Ixx2} \sin[\theta_2[t]]^2 + \text{Ixx3} \sin[\theta_2[t] + \theta_3[t]]^2 & 0 & 0 \\ 0 & \text{Izz2} + \text{Izz3} + \text{lc2}^2 m_2 + (12^2 + \text{lc3}^2) m_3 + 2 \cdot 12 \cdot \text{lc3} m_3 \cos[\theta_3[t]] & \text{Izz3} + \text{lc3}^2 m_3 + 12 \cdot \text{lc3} m_3 \cos[\theta_3[t]] \\ 0 & \text{Izz3} + \text{lc3}^2 m_3 + 12 \cdot \text{lc3} m_3 \cos[\theta_3[t]] & \text{Izz3} + \text{lc3}^2 m_3 \end{pmatrix}$$

Figure 4: symbolic inertia matrix $M(q)$ from Mathematica, shown column by column: top - first column, middle - second column, bottom - third column.

To better understand, the inertia matrix is

$$M(q) = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{12} & M_{22} & M_{23} \\ M_{13} & M_{23} & M_{33} \end{bmatrix} \quad (2.16)$$

where the independent terms are

$$\begin{aligned} M_{11} &= I_{yy1} + I_{yy2} c_2^2 + m_2 l_{c2}^2 c_2^2 + I_{yy3} c_{23}^2 \\ &\quad + m_3 (l_2 c_2 + l_{c3} c_{23})^2 + I_{xx2} s_2^2 + I_{xx3} s_{23}^2 \\ M_{12} &= 0 \\ M_{13} &= 0 \\ M_{22} &= I_{zz2} + I_{zz3} + m_2 l_{c2}^2 + m_3 (l_2^2 + l_{c3}^2) + 2 m_3 l_2 l_{c3} c_3 \\ M_{23} &= I_{zz3} + m_3 l_{c3}^2 + m_3 l_2 l_{c3} c_3 \\ M_{33} &= I_{zz3} + m_3 l_{c3}^2 \end{aligned} \quad (2.17)$$

The matrix is symmetric, as expected for a physical inertia matrix. The first generalized coordinate θ_1 is dynamically decoupled from θ_2 and θ_3 in the inertia matrix, which is why the terms M_{12} and M_{13} are zero. The coupling between the second and third joints appears through M_{23} , which depends on the relative angle θ_3 .

2.6.2. Coriolis and Centrifugal Matrix $C(q, \dot{q})$

After calculating the inertia matrix $M(q)$, we calculated the Coriolis and centrifugal matrix. In class, we saw that the Coriolis matrix can be written using the inertia matrix and its derivatives as

$$C(q, \dot{q}) = \dot{M} - \frac{1}{2} \begin{bmatrix} \dot{q}^T \frac{\partial M}{\partial q_1} \\ \dot{q}^T \frac{\partial M}{\partial q_2} \\ \vdots \\ \dot{q}^T \frac{\partial M}{\partial q_n} \end{bmatrix} \quad (2.18)$$

For our robot, the system has three generalized coordinates therefore, the expression becomes

$$C(q, \dot{q}) = \dot{M} - \frac{1}{2} \begin{bmatrix} \dot{q}^T \frac{\partial M}{\partial \theta_1} \\ \dot{q}^T \frac{\partial M}{\partial \theta_2} \\ \dot{q}^T \frac{\partial M}{\partial \theta_3} \end{bmatrix} \quad (2.19)$$

In Mathematica, we first calculated the time derivative of the inertia matrix $\dot{M}$ and we calculated the partial derivatives of M with respect to each generalized coordinate

```
MDot = D[M, t];
MTag = {D[M, q[[1]]], D[M, q[[2]]], D[M, q[[3]]]};
```

Figure 5: Mathematica calculation of $\dot{M}$ and the partial derivatives of $M(q)$

The Coriolis Matrix implemented in Mathematica as

$$\mathbf{CMat} = \mathbf{MDot} - \frac{1}{2} \{ \text{Transpose}[\mathbf{qDot}] . \mathbf{MTag}[[1]], \text{Transpose}[\mathbf{qDot}] . \mathbf{MTag}[[2]], \text{Transpose}[\mathbf{qDot}] . \mathbf{MTag}[[3]] \}$$

Figure 6: The Coriolis Matrix input in Mathematica

$$\begin{pmatrix} ((I_{xx2} - I_{yy2} - l_{c2}^2 m_2 - l_2^2 m_3) \sin[2 \theta_2[t]] + (I_{xx3} - I_{yy3} - l_{c3}^2 m_3) \sin[2 (\theta_2[t] + \theta_3[t])] - 2 l_2 l_{c3} m_3 \sin[2 \theta_2[t] + \theta_3[t]]) \theta_2'[t] - 2 (l_2 l_{c3} m_3 \cos[\theta_2[t]] + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \cos[\theta_2[t] + \theta_3[t]]) \sin[\theta_2[t] + \theta_3[t]] \theta_3'[t] \\ \frac{1}{2} ((-I_{xx2} + I_{yy2} - l_{c2}^2 m_2 - l_2^2 m_3) \sin[2 \theta_2[t]] + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \sin[2 (\theta_2[t] + \theta_3[t])] + 2 l_2 l_{c3} m_3 \sin[2 \theta_2[t] + \theta_3[t]]) \theta_1'[t] \\ (l_2 l_{c3} m_3 \cos[\theta_2[t]] + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \cos[\theta_2[t] + \theta_3[t]]) \sin[\theta_2[t] + \theta_3[t]] \theta_1'[t] \\ \begin{matrix} \theta & \theta \\ -2 l_2 l_{c3} m_3 \sin[\theta_3[t]] \theta_3'[t] & -l_2 l_{c3} m_3 \sin[\theta_3[t]] \theta_3'[t] \\ \frac{1}{2} l_2 l_{c3} m_3 \sin[\theta_3[t]] (2 \theta_2'[t] - \theta_3'[t]) & \frac{1}{2} l_2 l_{c3} m_3 \sin[\theta_3[t]] \theta_2'[t] \end{matrix} \end{pmatrix}$$

Figure 7: The Coriolis Matrix output in Mathematica

The symbolic result is

$$\mathcal{C}(q, \dot{q}) = \begin{bmatrix} C_{11} & 0 & 0 \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix} \quad (2.20)$$

where the non-zero terms are:

$$\begin{aligned} C_{11} &= [(I_{xx2} - I_{yy2} - l_{c2}^2 m_2 - l_2^2 m_3) \sin(2\theta_2) \\ &\quad + (I_{xx3} - I_{yy3} - l_{c3}^2 m_3) \sin(2(\theta_2 + \theta_3)) \\ &\quad - 2l_2 l_{c3} m_3 \sin(2\theta_2 + \theta_3)] \dot{\theta}_2 \\ &\quad - 2[l_2 l_{c3} m_3 \cos(\theta_2) + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \cos(\theta_2 + \theta_3)] \\ &\quad \cdot \sin(\theta_2 + \theta_3) \dot{\theta}_3 \\ C_{21} &= \frac{1}{2} [(-I_{xx2} + I_{yy2} + l_{c2}^2 m_2 + l_2^2 m_3) \sin(2\theta_2) \\ &\quad + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \sin(2(\theta_2 + \theta_3)) \\ &\quad + 2l_2 l_{c3} m_3 \sin(2\theta_2 + \theta_3)] \dot{\theta}_1 \\ C_{22} &= -2l_2 l_{c3} m_3 \sin(\theta_3) \dot{\theta}_3 \\ C_{23} &= -l_2 l_{c3} m_3 \sin(\theta_3) \dot{\theta}_3 \\ C_{31} &= [l_2 l_{c3} m_3 \cos(\theta_2) + (-I_{xx3} + I_{yy3} + l_{c3}^2 m_3) \cos(\theta_2 + \theta_3)] \\ &\quad \cdot \sin(\theta_2 + \theta_3) \dot{\theta}_1 \\ C_{32} &= \frac{1}{2} l_2 l_{c3} m_3 \sin(\theta_3) (2\dot{\theta}_2 - \dot{\theta}_3) \\ C_{33} &= \frac{1}{2} l_2 l_{c3} m_3 \sin(\theta_3) \dot{\theta}_2 \end{aligned} \quad (2.21)$$

2.6.3. Gravity Vector

The gravity vector describes the generalized torques created by the weight of the robot links. In this model, the Y -axis is defined as the vertical axis, and therefore the gravitational acceleration vector is

$$g_{vec} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix} \quad (2.22)$$

The gravitational force acting on link i is $m_i g_{vec}$. To express this force in the generalized coordinates, it is projected using the transpose of the linear Jacobian of the link center of mass. Therefore, the gravity vector is calculated as

$$G(q) = - \sum_{i=1}^3 m_i J_{L_i}^T g_{vec} \quad (2.23)$$

where J_{L_i} is the linear Jacobian of the center of mass of link i . The minus sign appears because the gravity vector in the equations of motion is written on the left-hand side of the motion equation. Using Mathematica, this expression was implemented symbolically as

```
gVec = {0, -g, 0};
Gvec = -Sum[m[[i]] x Transpose[JL[[i]]].gVec, {i, 1, 3}] // FullSimplify;
MatrixForm[Gvec] // FullSimplify
```

Figure 8: The gravity Matrix input in Mathematica

and the resulting gravity vector is

$$\begin{pmatrix} 0 \\ g ((l_2 m_2 + l_2 m_3) \cos[\theta_2[t]] + l_3 m_3 \cos[\theta_2[t] + \theta_3[t]]) \\ g l_3 m_3 \cos[\theta_2[t] + \theta_3[t]] \end{pmatrix}$$

Figure 9: The gravity Matrix output in Mathematica

The symbolic result is

$$G(q) = \begin{bmatrix} 0 \\ g(m_2 l_{c2} c_2 + m_3 l_2 c_2 + m_3 l_{c3} c_{23}) \\ g m_3 l_{c3} c_{23} \end{bmatrix} \quad (2.24)$$

The first component is zero because θ_1 represents rotation around the vertical Y -axis. This motion does not change the height of the centers of mass, and therefore gravity does not create a direct generalized torque in the θ_1 coordinate.

3. Numerical Integration of the Equations

After deriving the equations of motion, the next step was to solve them numerically in order to simulate the robot motion. Since this equation is a second-order nonlinear differential equation, it must be reduced to a first-order system before numerical integration. The state vector is defined as

$$x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} \quad (3.1)$$

The time derivative of the state vector is

$$\dot{x} = \begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} \quad (3.2)$$

From the equations of motion, the acceleration vector can be isolated as

$$\ddot{q} = M(q)^{-1}[\tau - C(q, \dot{q})\dot{q} - G(q)] \quad (3.3)$$

Therefore, the first-order system used for the numerical integration is

$$\dot{x} = \begin{bmatrix} \dot{q} \\ M(q)^{-1}[\tau - C(q, \dot{q})\dot{q} - G(q)] \end{bmatrix} \quad (3.4)$$

This form allows the robots dynamics to be integrated using the ODE45 numerical solver. At each time step, the current state $x(t)$ is used to extract $q(t)$ and $\dot{q}(t)$. Then, the matrices $M(q)$, $C(q, \dot{q})$, and $G(q)$ are evaluated, and the joint accelerations $\ddot{q}(t)$ are calculated. These accelerations are returned together with the joint velocities as the derivative of the state vector.

4. Initial Conditions and Zero-Input Simulation Setup

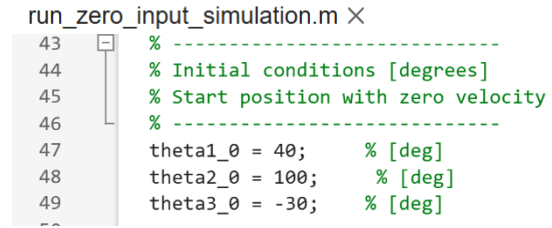
The initial conditions for the zero-input simulation are defined in the main MATLAB file:

Main File: Zero Input Simulation

File: run_zero_input_simulation.m.

Section: Initial conditions

Lines: 43–49



```
run_zero_input_simulation.m ×
43 % -----
44 % Initial conditions [degrees]
45 % Start position with zero velocity
46 % -----
47 theta1_0 = 40; % [deg]
48 theta2_0 = 100; % [deg]
49 theta3_0 = -30; % [deg]
```

Figure 10: Snapshot from the code – define initial conditions

After deriving the dynamic model and reducing the equations of motion to first-order form, the model was simulated in MATLAB in order to examine the natural motion of the robotic arm. In this simulation no control input was applied, and therefore the torque vector was set to $\tau = [0 \ 0 \ 0]^T$.

The robot was released from an initial configuration with zero joint velocities $\dot{q}(0) = [0 \ 0 \ 0]^T$. This means that the robot starts from rest, and its motion is generated only by the gravitational forces and by the dynamic coupling between the links. In the MATLAB implementation, the state was defined in degrees for readability, while the dynamic calculations were performed in radians. At each time step, the current state the matrices $M(q)$, $C(q, \dot{q})$, and $G(q)$ were evaluated, and the acceleration vector was calculated from

$$\ddot{q} = M^{-1}(q)(-C(q, \dot{q})\dot{q} - G(q)). \quad (4.1)$$

The simulation using ode45, with a time step of 0.01 seconds. The solver tolerances were chosen to be small in order to obtain a smooth and accurate motion. The output of the simulation includes a stick-figure animation of the robot motion and two graphs that will be discussed later in this chapter.

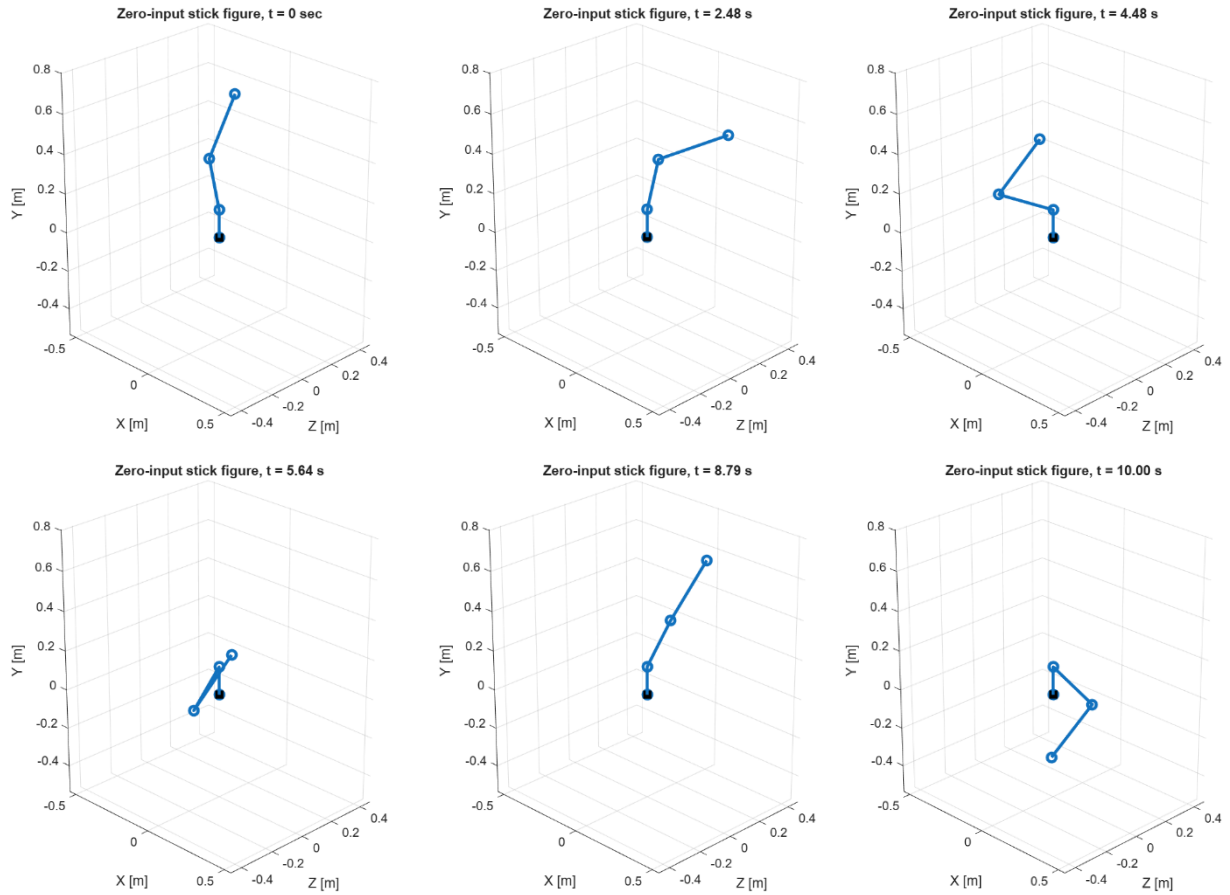


Figure 11: Zero-Input Stick-Figure Motion of the 3-DOF Robot Arm from Rest

The purpose of this simulation is to verify that the derived dynamic model produces a physically reasonable passive response. Since no damping or control torque was added, the system is expected to behave like an ideal conservative mechanical system. Therefore, the robot does not converge to a fixed position. Instead, the links move under gravity, and the energy is exchanged between potential and kinetic energy.

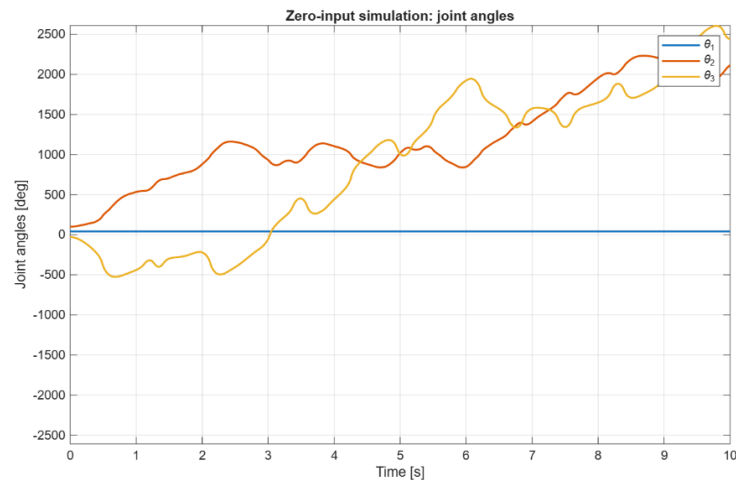


Figure 12: Time dependence Joint Velocities

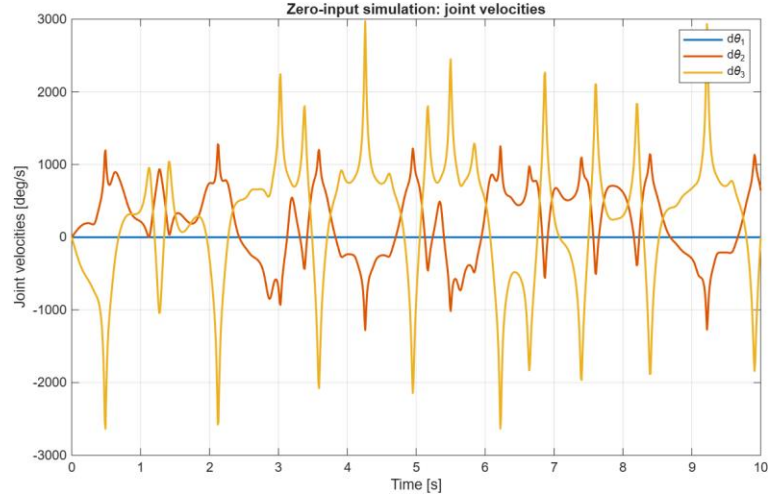


Figure 13: Zero-input simulation: joint velocities over time

The joint-angle graph in Figure 12 shows the corresponding motion of $\theta_1, \theta_2, \theta_3$. The angles are not limited to the range 0° – 360° , since the simulation represents continuous revolute-joint motion without joint limits. Therefore, values larger than 360° describe accumulated rotation and do not indicate an error in the simulation. The joint-velocity graph in Figure 13 shows the evolution of $\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3$ overtime. As defined in the initial conditions, all joint velocities start from zero. After the simulation begins, the velocities change due to the gravitational torques acting on the robot and the dynamic coupling between the links.

5. Control Using a Potential Function

In this chapter, a nonlinear control strategy based on an artificial potential function is employed. The basic idea is to construct a scalar potential function whose minimum corresponds to the desired robot configuration. The controller generates joint torques that drive the manipulator toward the minimum of this potential, similarly to the motion of a particle moving in a potential field. Since the considered task does not involve obstacle avoidance, the potential field consists solely of an attractive component, and the desired configuration is defined as the unique global minimum of the potential function. To ensure convergence and suppress oscillations, a damping term proportional to the joint velocities is added. The stability of the closed-loop system is analyzed using Lyapunov theory, where the total mechanical energy of the system serves as a Lyapunov candidate function. By showing that this energy continuously decreases over time, asymptotic convergence of the manipulator toward the desired equilibrium configuration can be established. This approach provides an intuitive energy-based framework for both motion control and stability analysis.

To implement this approach, we define desired configuration q_d .

$$q_d = \begin{bmatrix} q_{1d} \\ q_{2d} \\ q_{3d} \end{bmatrix} \quad (5.1)$$

5.1. Artificial Potential Function

The artificial potential function is strictly positive everywhere except at the desired configuration, where it reaches its unique global minimum and equal zero. This function is defined:

$$U(q) = U_g + U_{att} \quad (5.2)$$

Where,

$$G = -\nabla U_g \quad (5.3)$$

$$U_{att} = \frac{1}{2} (q - q_d)^T K_{att} (q - q_d) \quad (5.4)$$

$$K_{att} = \begin{bmatrix} k1 & 0 & 0 \\ 0 & k2 & 0 \\ 0 & 0 & k3 \end{bmatrix} \quad (5.5)$$

U_{att} implemented in Mathematica as

$$\mathbf{Uatt} = \frac{1}{2} \text{Transpose}[\mathbf{q} - \mathbf{qd}] . \mathbf{Katt} . (\mathbf{q} - \mathbf{qd}) // \text{FullSimplify}$$

Figure 14: U_{att} input in Mathematica

$$\frac{1}{2} (k_1 (\theta_{1d} - \theta_1[t])^2 + k_2 (\theta_{2d} - \theta_2[t])^2 + k_3 (\theta_{3d} - \theta_3[t])^2)$$

Figure 15: U_{att} output in Mathematica

The attractive potential function is chosen as a function of the joint-space error. This choice guarantees that the potential energy is strictly positive for every configuration different from the desired one and reaches zero only at $q = q_d$. Consequently, the desired configuration represents the unique global minimum of the potential field, ensuring that the generated torques always act to reduce the position error. The diagonal gains k_1 , k_2 , and k_3 determine the strength of the attractive field associated with each joint. Increasing a gain increases the restoring torque generated by the corresponding joint error and therefore accelerates convergence toward the desired configuration.

5.2. Artificial Torque Generation

The attractive torque vector is obtained as the negative gradient of the potential function. Therefore, the generated torques always point toward the desired configuration and decrease the potential energy of the system. A damping term is introduced to dissipate kinetic energy and prevent oscillatory behavior near the equilibrium point.

$$\tau(q) = -\nabla U(q) - K_d \dot{q} = G - \nabla U_{att}(q) - K_d \dot{q} \quad (5.6)$$

Where,

$$K_d = \begin{bmatrix} d1 & 0 & 0 \\ 0 & d2 & 0 \\ 0 & 0 & d3 \end{bmatrix} \quad (5.7)$$

τ implemented in Mathematica as

```
Kd = DiagonalMatrix[{d1, d2, d3}];
GradU = Grad[Uatt, {q[[1]], q[[2]], q[[3]]}] // FullSimplify
 $\tau$  = Gvec - GradU - Kd.qDot // FullSimplify
```

Figure 16: τ input in Mathematica

$$\begin{pmatrix} k_1 \theta_{1d} - k_1 \theta_1[t] - d_1 \theta_1'[t] \\ k_2 \theta_{2d} + g (l_{c2} m_2 + l_2 m_3) \cos[\theta_2[t]] + g l_{c3} m_3 \cos[\theta_2[t] + \theta_3[t]] - k_2 \theta_2[t] - d_2 \theta_2'[t] \\ k_3 \theta_{3d} + g l_{c3} m_3 \cos[\theta_2[t] + \theta_3[t]] - k_3 \theta_3[t] - d_3 \theta_3'[t] \end{pmatrix}$$

Figure 17: τ output in Mathematica

The resulting control torque consists of three contributions. The first contribution is the gravity compensation term $G(q)$, which counteracts the manipulator weight. The second contribution is the attractive term generated by the artificial potential function, which drives each joint toward its desired position. The third contribution is the damping term $-K_d \dot{q}$, which dissipates kinetic energy and suppresses oscillations near the equilibrium point. Together, these terms create stable closed-loop behavior that guides the manipulator toward the desired configuration.

6. Controlled Motion Simulation

This chapter presents the performance of the proposed controller when applied to the dynamic model of the 3-DOF robotic arm. Starting from an initial configuration with zero joint velocities, the robot is required to reach a predefined target position in Cartesian space. The target position is converted into a desired joint configuration through inverse kinematics, and the controller generates the required joint torques to drive the system toward this goal.

The simulation results are used to evaluate the convergence of the joint variables, the end-effector tracking accuracy, and the overall behavior of the robot during the maneuver. In addition, stick-figure visualizations are presented to provide a qualitative representation of the robot motion throughout the simulation.

6.1. Simulation Setup

The closed-loop simulations were performed using the MATLAB script `run_potential_control_simulation.m`. Running this script automatically initializes the robot parameters, generates the dynamic model, computes the desired joint-space target through inverse kinematics, and executes the numerical simulation.

The simulation starts from the initial joint configuration $\theta_0 = [50, 50, -20]$, with zero initial joint velocities. At the beginning of the simulation, the user is requested to enter a desired Cartesian target position in the form $[x, y, z]$ expressed in meters. If no input is provided, a predefined target point is used. To ensure that the requested target is physically reachable, the program performs a workspace validation before executing the inverse kinematics algorithm. The manipulator workspace is approximated as a spherical shell centered at $[0, 0.144, 0]$ meters, which corresponds to the shoulder joint location. The admissible target region is bounded by radius r with a range of 0.083 – 0.605 meters, where r denotes the distance between the target point and the workspace center.

If the user enters a target position outside the reachable workspace, the simulation is terminated and the following message is displayed: “Cartesian target is outside the reachable workspace.”

For valid target positions, the inverse kinematics routine automatically computes the corresponding joint-space target configuration. The branch that is closest to the initial robot configuration is selected and used as the reference for the controller.

The equations of motion are numerically integrated using MATLAB's ode45 solver. The simulation terminates either when the specified final time is reached or when the robot successfully reaches the target configuration according to predefined position and velocity tolerances.

6.2. Controller Parameters

The control law used throughout the simulations is the artificial potential-field controller developed in Chapter 5. Therefore, the mathematical formulation of the controller, including the potential function, torque generation mechanism, damping term, and gravity compensation, will not be repeated here.

For all simulations, the controller gains were selected to achieve an approximately critically damped response with a desired settling time of 2 seconds for each joint. The corresponding stiffness and damping gains were automatically computed using the manipulator inertia matrix evaluated at the target configuration. In addition, actuator torques were limited to ± 20 [N · m] to avoid unrealistically large control actions.

The following sections present the results obtained for one representative simulation scenario. Additional simulation cases, including different target positions and corresponding motion trajectories, are provided in the accompanying Google Drive folder together with the source code, videos, and supplementary materials.

6.3. Joint-Space Response

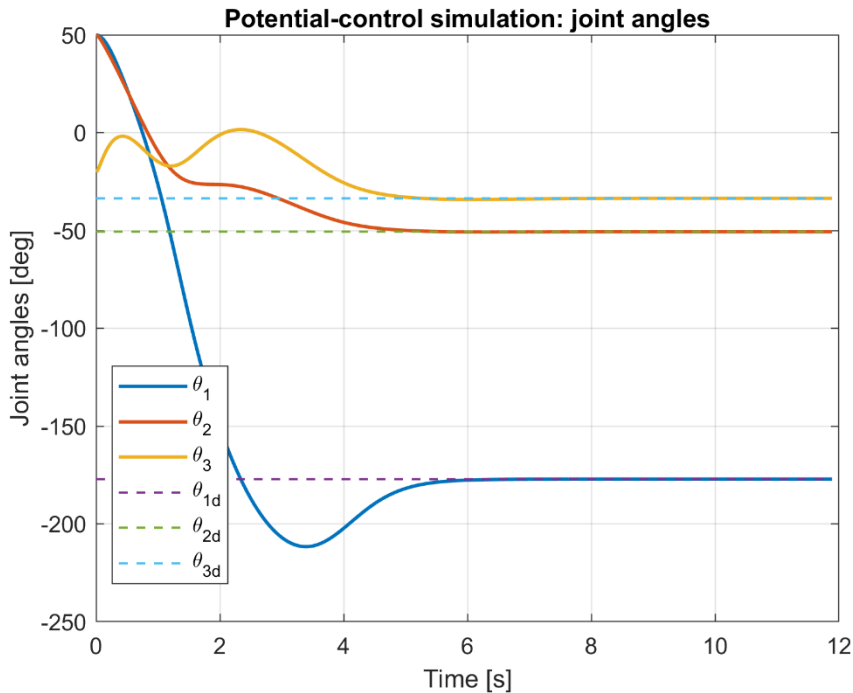


Figure 17: Joint space response

Figure 17 presents the evolution of the three joint angles during the closed-loop simulation. The manipulator starts from the initial configuration θ_0 , and is commanded to reach a Cartesian target position of $[0.2, 0.4, 0.01]$. The corresponding desired joint configuration, obtained through inverse kinematics, is shown by the dashed lines in the figure.

It can be observed that all three joints successfully converge with their desired values. Joint 1 undergoes the largest motion and therefore exhibits the most pronounced transient response. Joints 2 and 3 follow smoother trajectories with smaller deviations from their final values. Although a moderate overshoot is visible during the transient phase, no sustained oscillations are observed, and the response remains well damped throughout the maneuver.

The results indicate that the controller can drive the manipulator from the initial configuration to the desired equilibrium point while maintaining stable closed-loop behavior. All joint variables approach their reference values within approximately 5–6 seconds and remain close to the target configuration for the remainder of the simulation.

6.4. Joint Velocity Response

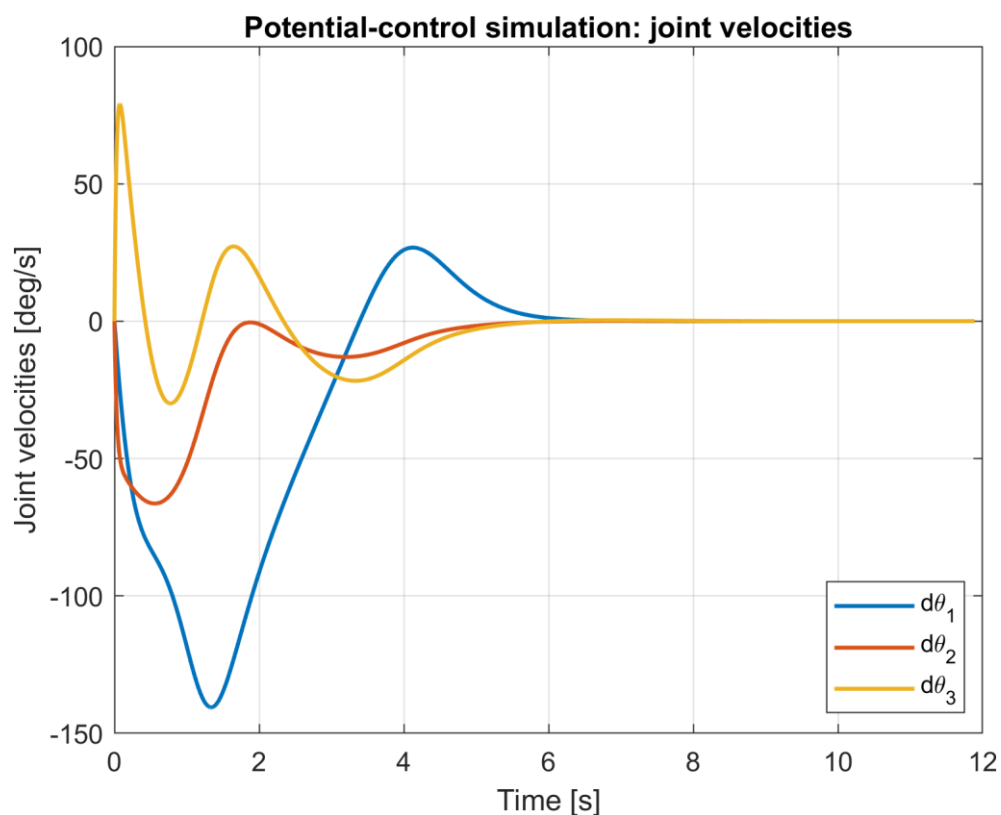


Figure 18: Joint velocity response

The joint velocity profiles obtained during the simulation are presented in Figure 18. Since the manipulator starts from rest, all joint velocities are initially equal to zero. As the controller drives the system toward the desired configuration, each joint accelerates according to the magnitude of its position error and the dynamic coupling between the robot links.

The largest velocity excursion is observed in Joint 1, which also experiences the largest angular displacement during the maneuver. This behavior is expected, as a larger change in position generally requires higher transient velocities in order to reach the target within the desired settling time. Joints 2 and 3 exhibit smaller velocity magnitudes and smoother responses throughout the simulation.

Although oscillatory behavior is visible during the transient phase, the amplitude of these oscillations decreases rapidly due to the damping component of the controller. As the robot approaches the desired equilibrium configuration, all joint velocities converge toward zero, indicating that the manipulator reaches a stationary final state.

The velocity response confirms the effectiveness of the damping term introduced in the control law. The controller provides sufficiently fast motion while preventing persistent oscillations and ensuring stable convergence to the desired configuration.

6.5. End-Effector Tracking Performance

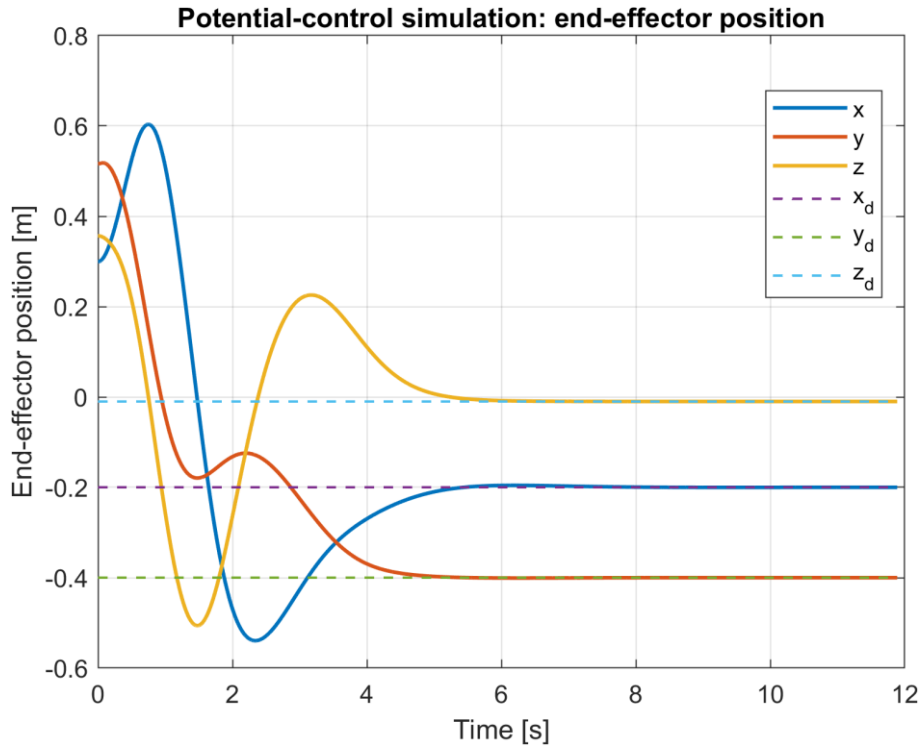


Figure 19: End-effector position

The ultimate objective of the controller is to drive the end-effector to the desired Cartesian target position. Therefore, in addition to the joint-space response, the motion of the end-effector was evaluated throughout the simulation.

Figure 19 presents the evolution of the Cartesian coordinates of the end-effector together with their corresponding reference values. Starting from its initial position, the end-effector follows a smooth trajectory toward the target point. The transient response includes moderate overshoot in some coordinates, which is expected due to the nonlinear dynamics of the manipulator and the relatively large change in configuration required to reach the desired position. Nevertheless, all three coordinates converge to their corresponding reference values and remain close to them for the remainder of the simulation.

The tracking performance can be evaluated more clearly through the Cartesian position error shown in Figure 20. The error is defined as the Euclidean distance between the current end-effector position and the desired target position. It can be observed that the error decreases continuously throughout the simulation, spanning several orders of magnitude before reaching values close to numerical precision.

Small variations in the error curve appear during the transient phase and correspond to the oscillatory behavior observed previously in the joint-space response. However, the overall trend remains strongly decreasing, indicating successful convergence toward the target. By the end of the simulation, the Cartesian position error is reduced to approximately 10^{-6} meters, demonstrating a high level of positioning accuracy.

These results confirm that the proposed controller successfully guides the manipulator to the desired Cartesian target while maintaining stable and accurate closed-loop behavior.

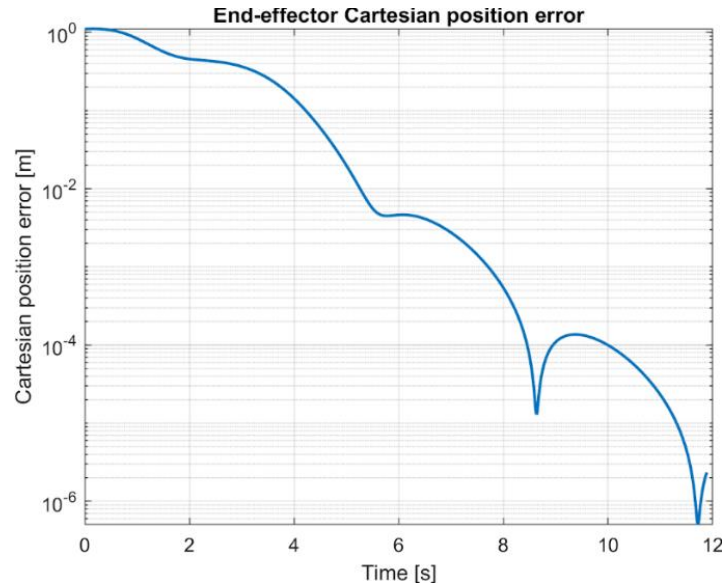


Figure 20: Cartesian position error

6.6. Control Effort

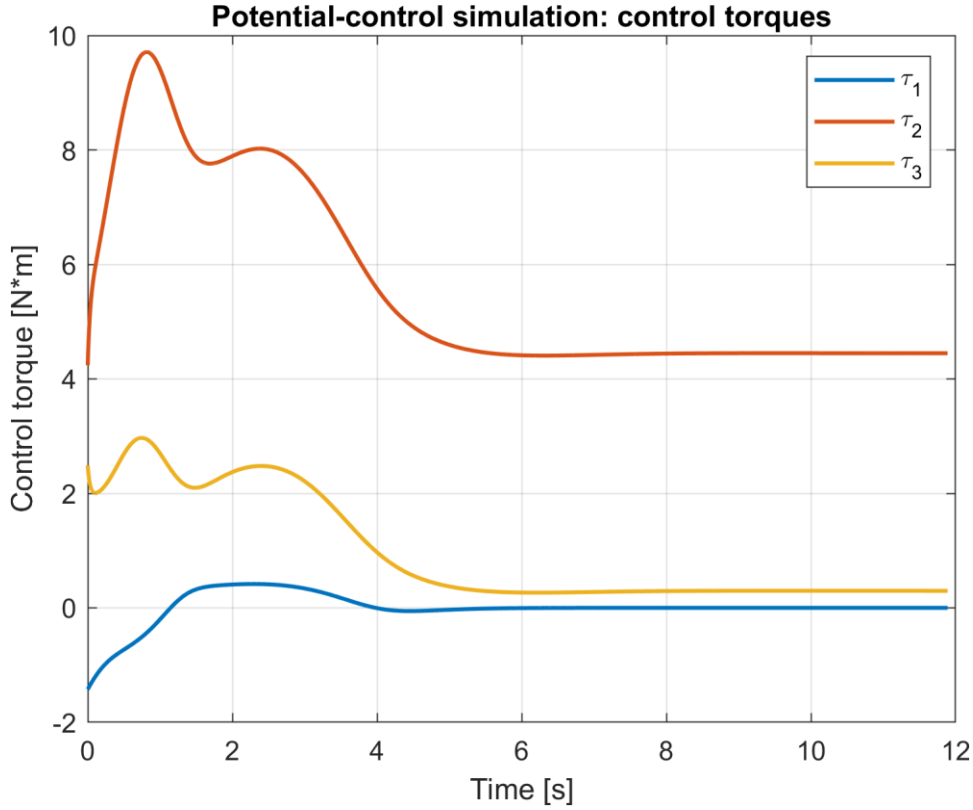


Figure 21: Control torque

The control torques generated by the controller during the simulation are presented in Figure 21. These torques represent the actuator effort required to drive the manipulator from its initial configuration to the desired target position while compensating for the effects of gravity and the robot dynamics.

During the initial phase of the motion, relatively large torques are generated in response to the significant position error between the current and desired configurations. As the manipulator approaches the target, the tracking error decreases and the control torques gradually settle to nearly constant values.

It can be observed that Joint 2 requires the largest control effort throughout the simulation, reaching a peak torque of approximately $10 \text{ [N}\cdot\text{m}]$. This behavior is expected because the second joint supports a significant portion of the manipulator mass and is strongly affected by gravitational loading. Joint 3 requires a smaller but still

noticeable torque, while the torque applied to Joint 1 remains comparatively low due to the rotational symmetry of the robot about the vertical axis.

The steady-state torque values observed after convergence correspond primarily to gravity compensation. Since the robot reaches a static equilibrium configuration with negligible joint velocities and position errors, the controller only needs to provide the torques required to balance the gravitational forces acting on the links.

Furthermore, all generated torques remain well below the imposed saturation limit of $\pm 20 \text{ [N} \cdot \text{m]}$. This indicates that the selected controller gains achieve the desired motion without requiring excessive actuator effort and that the simulated maneuver remains physically feasible for the considered robot model.

6.7. Potential Energy Evolution

As discussed in Chapter 5, the controller is based on an artificial attractive potential function whose minimum corresponds to the desired joint configuration. Consequently, successful convergence of the manipulator should be accompanied by a continuous reduction of the potential energy throughout the simulation.

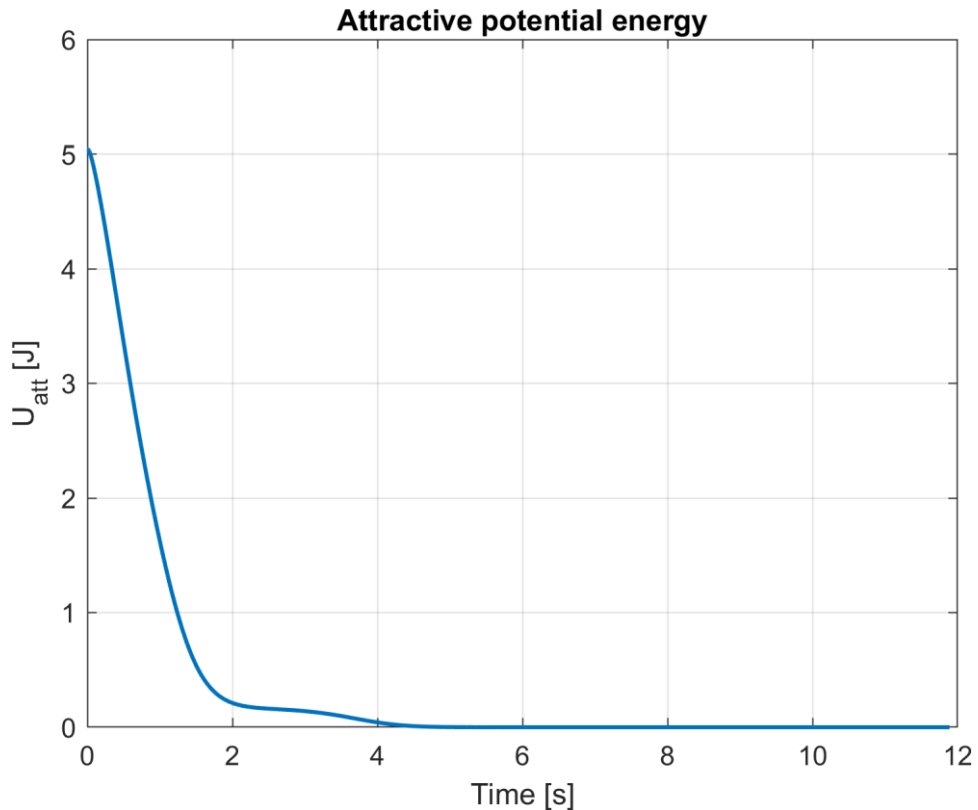


Figure 30: Attractive potential energy

The evolution of the attractive potential energy is presented in Figure 30. At the beginning of the simulation, the robot is located far from the desired configuration, resulting in a relatively high potential energy value. As the controller drives the joints

toward their reference positions, the configuration error decreases and the potential energy is gradually reduced.

The energy curve exhibits a generally decreasing trend and approaches zero as the manipulator reaches the target configuration. Small variations can be observed during the transient phase, reflecting the oscillatory behavior previously identified in the joint and Cartesian responses. However, these fluctuations diminish over time as the system converges toward equilibrium.

The final near-zero value of the potential energy indicates that the configuration error has become negligible and that the manipulator has reached the minimum of the artificial potential field. This behavior is consistent with the theoretical principle of the controller, where the robot motion can be interpreted as moving "downhill" along the potential landscape until the desired configuration is achieved.

The observed energy evolution therefore provides an additional indication of the stability and convergence properties of the proposed control approach.

6.8. Stick-Figure Motion

To complement the numerical results presented in the previous sections, a stick-figure animation of the manipulator motion was generated as part of the simulation framework. The animation provides a visual representation of the robot trajectory from the initial configuration to the desired target position and allows qualitative evaluation of the controller performance.

The animation confirms the conclusions obtained from the joint-space and Cartesian-space analyses. The manipulator follows a smooth trajectory toward the target configuration and reaches the desired position without exhibiting unstable behavior or excessive oscillations.

Due to the dynamic nature of the visualization, the complete stick-figure simulation is provided as a video in the accompanying Google Drive folder together with the source code and additional simulation scenarios. The video allows direct observation of the robot motion throughout the entire maneuver and serves as a visual validation of the controller performance.

6.9. Discussion of Results

The simulation results demonstrate that the proposed artificial potential-field controller successfully drives the manipulator from the initial configuration to the desired Cartesian target position. Both the joint-space and Cartesian-space responses show

stable convergence toward the reference values, while the velocity profiles confirm that the system reaches a stationary equilibrium state at the end of the maneuver.

The Cartesian tracking results indicate a continuous reduction of the position error throughout the simulation, eventually reaching values close to numerical precision. This confirms that the inverse kinematics solution and the control law operate consistently to achieve accurate end-effector positioning.

The control torque profiles show that the required actuator effort remains within the predefined limits during the entire maneuver. Although relatively large torques are generated during the transient phase, all values remain below the saturation threshold, indicating that the motion is physically feasible for the considered robot model.

The evolution of the attractive potential energy is consistent with the theoretical behavior described in Chapter 5. As the robot approaches the desired configuration, the potential energy decreases toward its minimum value, providing an additional indication of successful convergence.

Overall, the results demonstrate that the proposed control approach provides accurate target acquisition, stable closed-loop behavior, and smooth robot motion. The combination of joint-space convergence, low final Cartesian error, bounded control effort, and successful visual validation through simulation confirms the effectiveness of the controller for point-to-point positioning tasks of the considered 3-DOF robotic manipulator.

